POLARIS
SCHOOL OF TECHNOLOGY

Weight Initialization in Neural Networks

Need for Weight Initialization, Xavier (Glorot) and He (Kaiming) Techniques

What is Weight Initialization?

Weight initialization means assigning initial values to the weights of a neural network before training begins.

For a neuron: z = w1x1 + w2x2 + ... + wnxn + b. Before the first forward pass, the values of the weights must be initialized.

A good initialization strategy affects whether the network learns effectively, convergence speed, vanishing gradients, exploding gradients, symmetry between neurons, and training stability.

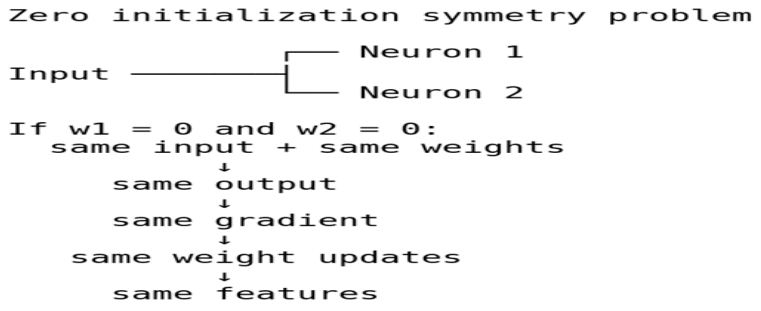
Why Do We Need Weight Initialization?

Training is an optimization process. Initially, the network does not know the correct weights, so initialization provides the starting point from which the optimizer begins learning.

$$W_{new} = W_{old} - \eta(\partial L/\partial W)$$

Therefore, initialization is the starting point of training.

Why Can't We Initialize All Weights to Zero?



If multiple neurons start with identical weights, they receive the same inputs, produce the same outputs, receive the same gradients, and continue to learn the same features. This is the symmetry problem.

Weights should generally not be initialized to zero. Biases, however, can commonly be initialized to zero.

Why Not Use Arbitrary Random Values?

Random initialization breaks symmetry, but arbitrary random values may be too small or too large.

- Weights that are too small can cause gradients to shrink through repeated multiplication, leading to vanishing gradients.
- Weights that are too large can cause gradients to grow through repeated multiplication, leading to exploding gradients.

Therefore, we need a principled method for choosing the scale of initial weights.

Main Goal of Good Weight Initialization

```
Input
  ↓
Layer 1
  ↓
Layer 2
  ↓
Layer 3
  ↓
Layer 4
  ↓
Output

Activations should not become
extremely small or extremely large.

Gradients should not vanish or explode.
```

A good initialization attempts to maintain a reasonable variance of activations as they move forward through the network and gradients as they move backward.

Xavier / Glorot Weight Initialization

Xavier initialization, also called Glorot initialization, was introduced by Xavier Glorot and Yoshua Bengio. It is particularly suitable for sigmoid and tanh activations.

Suppose a layer has nin input neurons and nout output neurons. Xavier chooses the weight scale using both nin and nout.

Xavier Normal Initialization

$$W \sim N(0, 2/(nin + nout))$$

$$\sigma = \sqrt{(2/(nin + nout))}$$

This makes the variance depend on both the number of input and output neurons.

Xavier Uniform Initialization

$$W \sim U(-a, a)$$

$$a = \sqrt{(6/(nin + nout))}$$

Thus, weights are sampled uniformly within the corresponding range.

Xavier Example

Suppose nin = 100 and nout = 50.

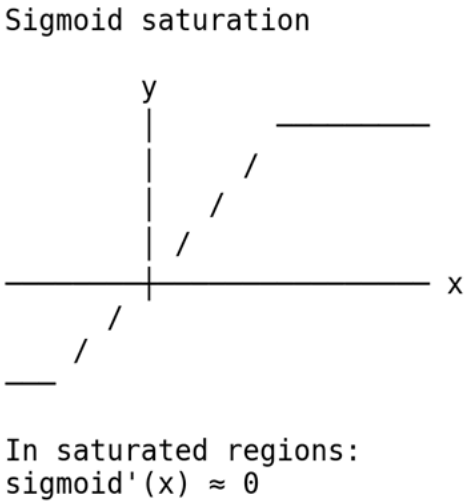
$$Var(W) = 2/(100 + 50) = 0.0133$$

$$\sigma \approx 0.115$$

So Xavier normal approximately samples weights from N(0, 0.0133).

Why is Xavier Good for Sigmoid and Tanh?

Sigmoid has derivative $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, with a maximum derivative of 0.25. Large inputs push sigmoid toward saturation, where its derivative becomes close to zero. Tanh can also saturate for large positive or negative inputs.



Xavier initialization tries to keep initial activations in a suitable range so that signal and gradient scales do not deteriorate too quickly.

He / Kaiming Weight Initialization

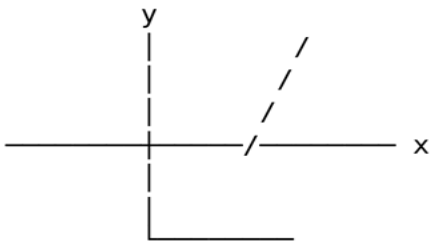
He initialization, also called Kaiming initialization, was introduced by Kaiming He and colleagues. It is particularly designed for ReLU and is commonly used with ReLU, Leaky ReLU, and PReLU.

Why Do We Need a Special Initialization for ReLU?

ReLU is f(x) = max(0, x). Negative inputs produce zero, which changes the variance of activations. He initialization compensates for this behavior.

ReLU

f(x) = max(0, x)



Negative input → 0
Positive input → x

He Normal Initialization

$$W \sim N(0, 2/nin)$$

$$\sigma = \sqrt{(2/nin)}$$

Unlike Xavier, He initialization primarily depends on the number of input neurons.

He Uniform Initialization

$$W \sim U(-a, a)$$

$$a = \sqrt{(6/nin)}$$

He Example

Suppose nin = 100.

$$Var(W) = 2/100 = 0.02$$

$$\sigma = \sqrt{0.02} \approx 0.141$$

So He normal approximately samples weights from N(0, 0.02).

Why is He Better for ReLU?

ReLU sets negative values to zero. He initialization compensates for the resulting reduction in activation variance and helps maintain the scale of activations and gradients across layers.

ReLU + He initialization

Xavier vs He

Feature	Xavier / Glorot	He / Kaiming
Main purpose	Maintain activation/gradient variance	Compensate for ReLU behavior
Best suited for	Sigmoid, Tanh	ReLU, Leaky ReLU, PReLU
Normal variance	2/(nin+nout)	2/nin
Uniform range	$\pm\sqrt{(6/(nin+nout))}$	$\pm\sqrt{(6/nin)}$
Depends on	Input + output neurons	Mainly input neurons
Common use	Networks using tanh/sigmoid	Modern CNNs/MLPs using ReLU

Easy Rule to Remember

Sigmoid / Tanh → Xavier

ReLU / Leaky ReLU / PReLU → He

Complete Conceptual Connection

```
Training concepts

Weight Initialization
        ↓
Initial weight scale
        ↓
Activation scale
        ↓
Backpropagation derivatives
        ↓
Gradient magnitude
        ↓
Vanishing / Exploding gradients
        ↓
Training and weight updates
```

Weight initialization determines the initial scale of weights. This affects activation scale and the derivatives encountered during backpropagation. Poor scaling can contribute to vanishing or exploding gradients, which in turn affects training and weight updates.

Summary

- Weight initialization provides the starting values for neural-network weights.
- Zero initialization of hidden-layer weights causes the symmetry problem.
- Arbitrary random initialization breaks symmetry but may still give poor gradient scales.
- Xavier/Glorot is mainly associated with sigmoid and tanh.
- He/Kaiming is mainly associated with ReLU and its variants.
- Xavier normal uses variance 2/(nin+nout).
- He normal uses variance 2/nin.
- Good initialization helps keep activations and gradients at useful scales.
- Initialization happens before optimization; it is not an optimizer itself.